MODULE 4

INTRODUCTION TO Hive

MODULE-4
Introduction to Hive: What is Hive, Hive Architecture, Hive data types, Hive file formats, Hive Query Language (HQL), RC File implementation, User Defined Function (UDF). **Introduction to Pig:** What is Pig, Anatomy of Pig, Pig on Hadoop, Pig Philosophy, Use case for Pig, Pig Latin Overview, Data types in Pig, Running Pig, Execution Modes of Pig, HDFS Commands, Relational Operators, Eval Function, Complex Data Types, Piggy Bank, User Defined Function, Pig Vs Hive. **TB1:** Ch 9: 9.1-9.6,9.8, Ch 10: 10.1 - 10.15, 10.22

CASE STUDY: RETAIL LOG PROCESSING

About the Company

TENTOTEN is a Retail Store which has a chain of hypermarkets in India. They have 250+ stores across 95 cities and towns. About 45,000+ people are working in **TENTOTEN**. **TENTOTEN** deals in a wide range of products including fashion apparels, food products, books, furniture, etc. Around 1500+ customers visit and/or purchase products every day from each of these stores.

Problem Scenario

The approximate size of **TENTOTEN** log datasets is 12 TB. Information about the various stores is stored in the form of semi-structured data. Traditional Business Intelligence (BI) tools are good when data is present in pre-defined schema and datasets are just several hundreds of gigabytes. But the **TENTOTEN** dataset is mostly log dataset, which does not conform to any particular schema. Querying such large dataset is difficult and immensely time consuming.

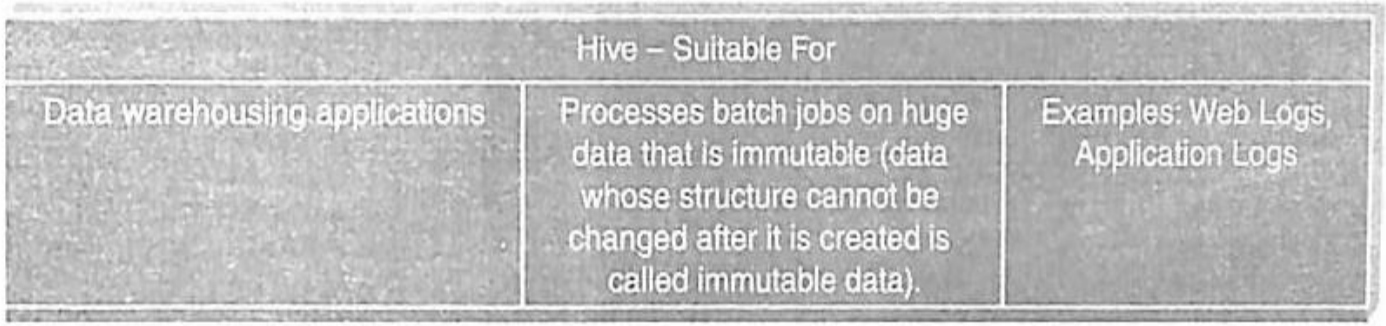
The challenges are:

1. Moving the log dataset to HDFS (Hadoop Distributed File System).
2. Performing analysis on HDFS data.

Hadoop MapReduce can be used to resolve these issues. However we will still have to deal with the below constraints:

1. Writing complex MapReduce jobs in Java can be tedious and error prone.
2. Joining across large datasets is quite tricky.

4.1 What is Hive ?



1. Summarization
2. Querying
3. Analysis

Figure 9.1 Hive – a data warehousing tool.

Facebook initially created Hive component to manage their ever-growing volumes of log data. Later Apache software foundation developed it as open-source and it came to be known as Apache Hive.

Hive makes use of the following:

1. HDFS for Storage.
2. MapReduce for execution.
3. Stores metadata/schemas in an RDBMS.

Hive provides HQL (Hive Query Language) or HiveQL which is similar to SQL. Hive compiles SQL queries into MapReduce jobs and then runs the job in the Hadoop Clust...

OLAP (Online Analytical Processing). Hive provides extensive data type functions and formats for data summarization and analysis.

Note:

1. Hive is not RDBMS.
2. It is not designed to support OLTP (Online Transaction Processing).
3. It is not designed for real-time queries.
4. It is not designed to support row-level updates.

2007: Hive originated at **Facebook**. At the time, Facebook was dealing with **huge amounts of data** and needed a way for analysts (who knew SQL but not Java or MapReduce) to easily query it.

Early 2008: Facebook engineers, including **Ashish Thusoo** and **Joydeep Sen Sarma**, developed Hive to make Hadoop more accessible through an SQL-like interface.

2008: Facebook contributed Hive to the **Apache Software Foundation**.

2009: Hive became an **Apache Incubator** project.

2010: Hive graduated to a **top-level Apache project**.

Over time, it became **one of the most popular tools** for querying large datasets in Hadoop systems. It enabled companies to use Hadoop without writing complex Java MapReduce code.

Today, Hive is still widely used in big data ecosystems, although newer technologies like **Presto**, **Trino**, and **Apache Spark SQL** sometimes offer faster performance for certain workloads.

4.2 History of Hive & recent releases

Figure 9.2 History of Hive.

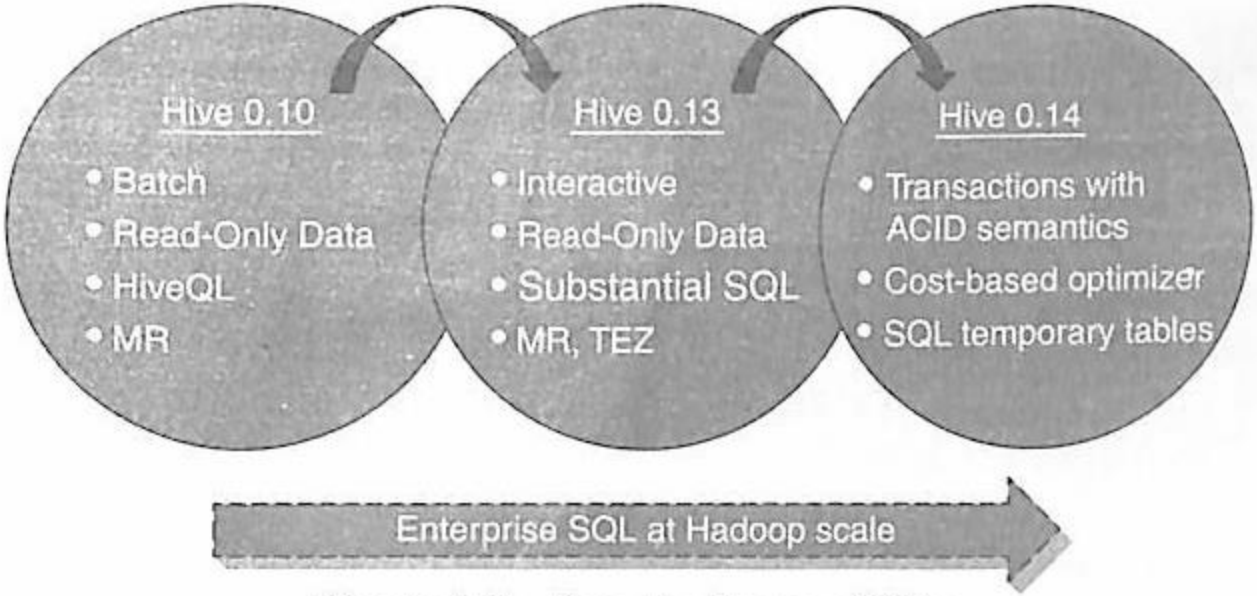
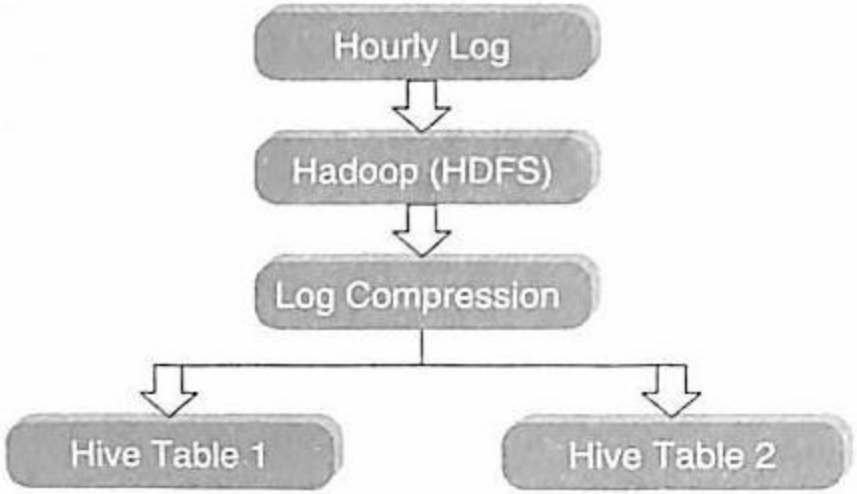


Figure 9.3 Recent releases of Hive.

Hive Features

1. It is similar to SQL.
2. HQL is easy to code.
3. Hive supports rich data types such as structs, lists and maps.
4. Hive supports SQL filters, group-by and order-by clauses.
5. Custom Types, Custom Functions can be defined.



9.1.4 Hive Data Units

1. **Databases:** The namespace for tables.
2. **Tables:** Set of records that have similar schema.
3. **Partitions:** Logical separations of data based on classification of given information as per specific attributes. Once hive has partitioned the data based on a specified key, it starts to assemble the records into specific folders as and when the records are inserted.
4. **Buckets (or Clusters):** Similar to partitions but uses hash function to segregate data and determines the cluster or bucket into which the record should be placed.

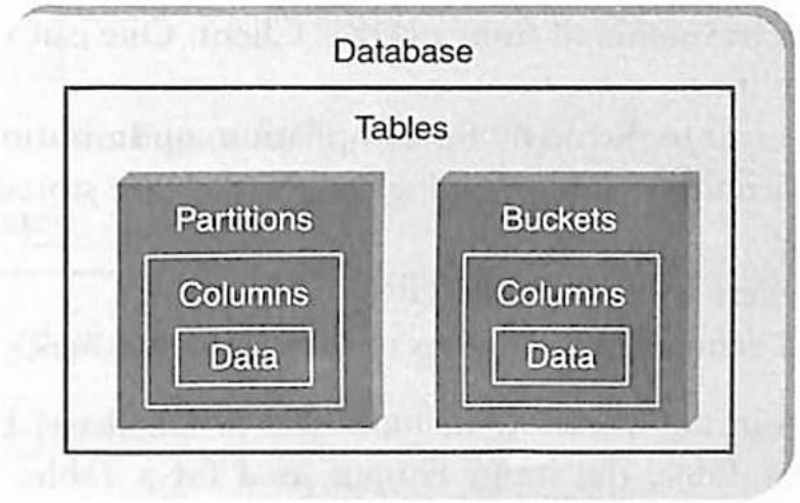


Figure 9.5 Data units as arranged in a Hive.

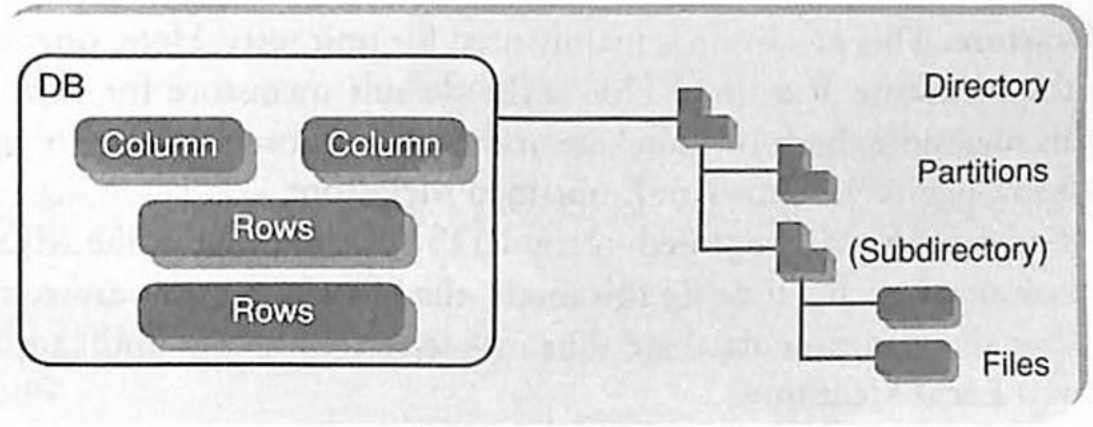


Figure 9.6 Semblance of Hive structure with database.

Hive Architecture

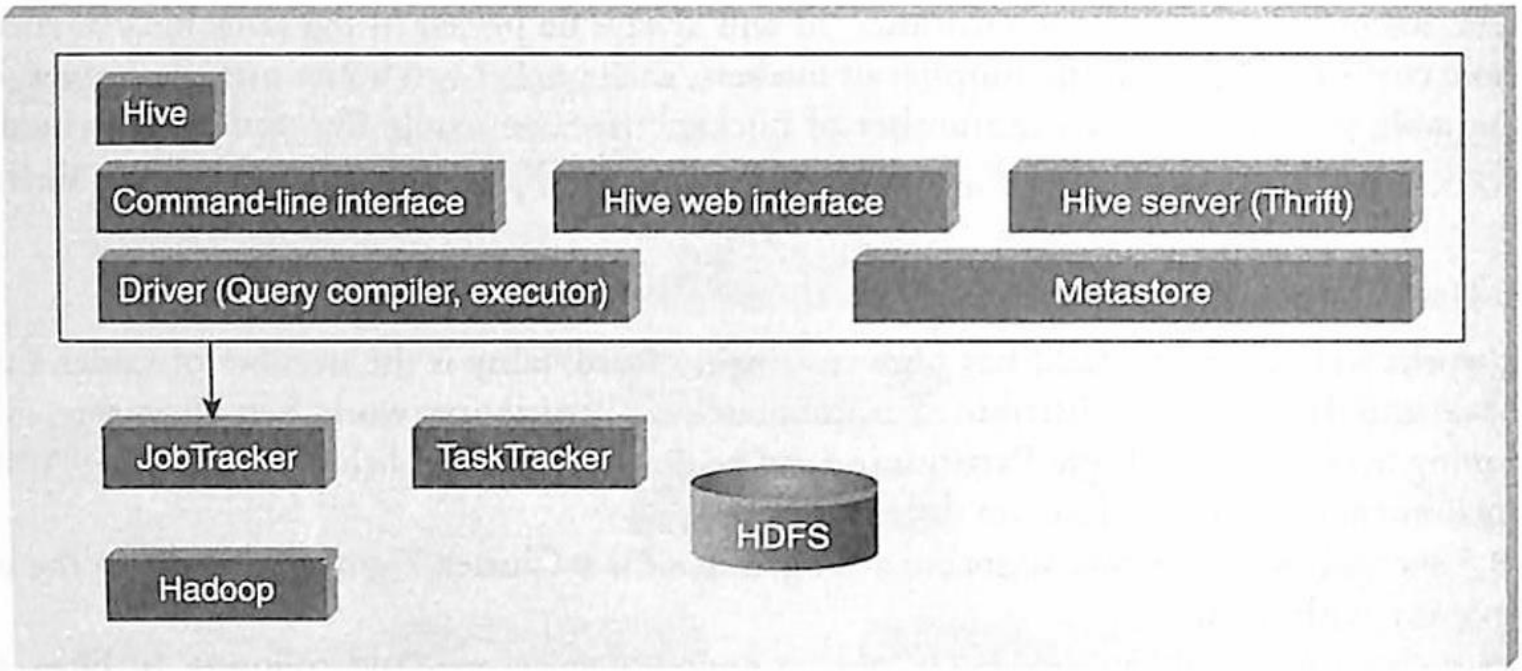


Figure 9.7 Hive architecture.

Figure 9.8 Embedded Metastore.

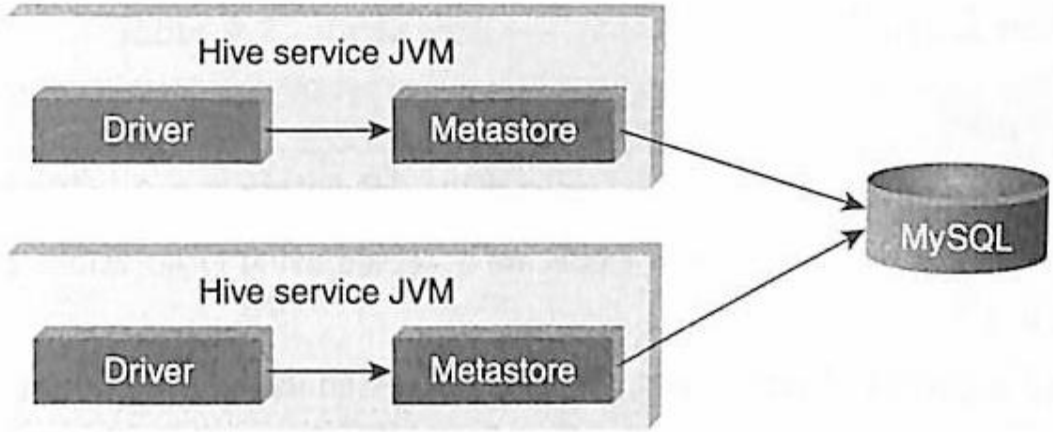


Figure 9.9 Local Metastore.

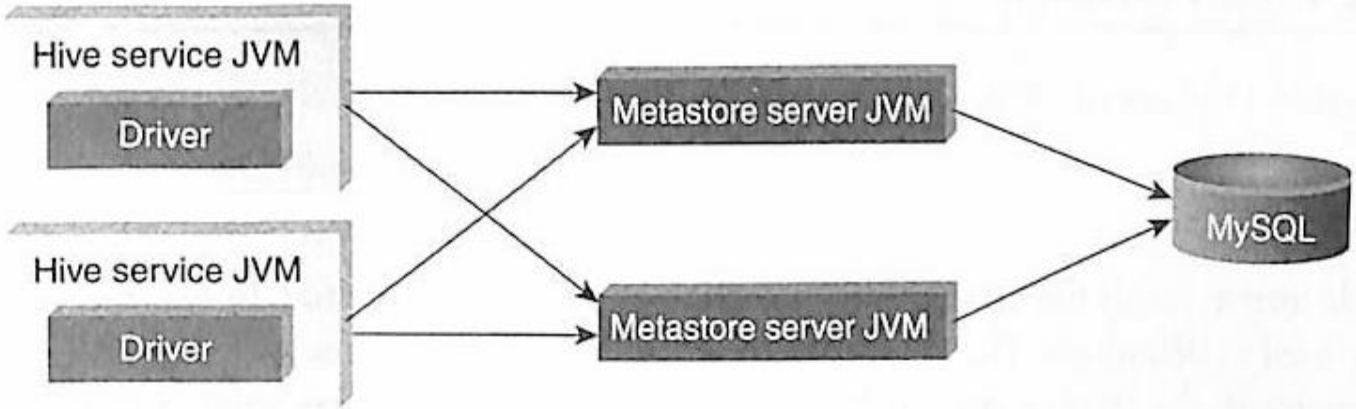


Figure 9.10 Remote Metastore.

Hive Data Types

Primitive Data Types

Numeric Data Type	
TINYINT	1-byte signed integer
SMALLINT	2-byte signed integer
INT	4-byte signed integer
BIGINT	8-byte signed integer
FLOAT	4-byte single-precision floating-point
DOUBLE	8-byte double-precision floating-point number

String Types	
STRING	
VARCHAR	Only available starting with Hive 0.12.0
CHAR	Only available starting with Hive 0.13.0

Strings can be expressed in either single quotes (') or double quotes (")

Miscellaneous Types	
BOOLEAN	
BINARY	Only available starting with Hive

Collection Data Types

Collection Data Types	
STRUCT	Similar to 'C' struct. Fields are accessed using dot notation. E.g.: struct('John', 'Doe')
MAP	A collection of key–value pairs. Fields are accessed using [] notation. E.g.: map('first', 'John', 'last', 'Doe')
ARRAY	Ordered sequence of same types. Fields are accessed using array index. E.g.: array('John', 'Doe')

Hive File Format

Text File

The default file format is text file. In this format, each record is a line in the file. In text file, different control characters are used as delimiters. The delimiters are ^A (octal 001, separates all fields), ^B (octal 002, separates the elements in the array or struct), ^C (octal 003, separates key–value pair), and \n. The term **field** is used when overriding the default delimiter. The supported text files are CSV and TSV. JSON or XML documents too can be specified as text file.

Sequential File

Sequential files are flat files that store binary key–value pairs. It includes compression support which reduces the CPU, I/O requirement.

Table 9.1 A table with four columns

C1	C2	C3	C4
11	12	13	14
21	22	23	24
31	32	33	34
41	42	43	44
51	52	53	54

Table 9.2 Table with two row groups

Row Group 1			
C1	**C2**	**C3**	**C4**
11	12	13	14
21	22	23	24
31	32	33	34

Row Group 2			
C1	**C2**	**C3**	**C4**
41	42	43	44
51	52	53	54

Table 9.3 Table in RCFile Format

Row Group 1	Row Group 2
11, 21, 31;	41, 51;
12, 22, 32;	42, 52;
13, 23, 33;	43, 53;
14, 24, 34;	44, 54;

HIVE QUERY LANGUAGE (HQL)

Hive query language provides basic SQL like operations. Here are few of the tasks which HQL can do easily.

1. Create and manage tables and partitions.
2. Support various Relational, Arithmetic, and Logical Operators.
3. Evaluate functions.
4. Download the contents of a table to a local directory or result of queries to HDFS directory.

DDL (Data Definition Language) Statements

These statements are used to build and modify the tables and other objects in the database. The DDL commands are as follows:

1. Create/Drop/Alter Database
2. Create/Drop/Truncate Table
3. Alter Table/Partition/Column
4. Create/Drop/Alter View
5. Create/Drop/Alter Index
6. Show
7. Describe

DML (Data Manipulation Language) Statements

These statements are used to retrieve, store, modify, delete, and update data in database. The DML commands are as follows:

1. Loading files into table.
2. Inserting data into Hive Tables from queries.

Starting Hive Shell

To start Hive, go to the installation path of Hive and type as below:

```
[root@volgalnx005 ~]# hive

Logging initialized using configuration in jar:file:/root/Desktop/VMDATA/Hive/hi
ve/lib/hive-common-0.14.0.jar!/hive-log4j.properties
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/root/Desktop/VMDATA/Hadoop/hadoop/share/hadoo
p/common/lib/slf4j-log4j12-1.7.5.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/root/Desktop/VMDATA/Hive/hive/lib/hive-jdbc-0
.14.0-standalone.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.slf4j.impl.Log4jLoggerFactory]
hive>
```

Objective: To create a database named "STUDENTS" with comments and database properties.

Act:

```
CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details'
WITH DBPROPERTIES ('creator' = 'JOHN');
```

Outcome:

```
hive> CREATE DATABASE IF NOT EXISTS STUDENTS COMMENT 'STUDENT Details' WITH DBPROPERTIES ('creato
r' = 'JOHN');
OK
Time taken: 0.536 seconds
hive>
```

Objective: To display a list of all databases.

Act:

```
SHOW DATABASES;
```

Outcome:

```
hive> SHOW DATABASES;
OK

students
Time taken: 0.082 seconds, Fetched: 22 row(s)
hive>
```

Objective: To describe a database.

Act:

DESCRIBE DATABASE STUDENTS;

Note: Shows only DB name, comment, and DB directory.

Outcome:

```
hive> DESCRIBE DATABASE STUDENTS;
OK
students        STUDENT Details hdfs://volgalnx010.ad.infosys.com:9000/user/hive/warehouse/studen
ts.db   root    USER
Time taken: 0.03 seconds, Fetched: 1 row(s)
hive>
```

Objective: To describe the extended database.

Act:

DESCRIBE DATABASE EXTENDED STUDENTS;

Note: Shows DB properties al

Outcome:

```
hive> DESCRIBE DATABASE EXTENDED STUDENTS;
OK
students        STUDENT Details hdfs://volgalnx010.ad.infosys.com:9000/user/hive/warehouse/studen
ts.db   root    USER    {creator=JOHN}
Time taken: 0.027 seconds, Fetched: 1 row(s)
hive>
```

Objective: To alter the database properties.

Act:

```
ALTER DATABASE STUDENTS SET DBPROPERTIES ('edited-by' = 'JAMES');
```

Note: We can use the "ALTER DATABASE" command to

- Assign any new (key, value) pairs into DBPROPERTIES.
- Set owner user or role to the Database.

In Hive, it is not possible to unset the DB properties.

Outcome:

```
hive> ALTER DATABASE STUDENTS SET DBPROPERTIES ('edited-by' = 'JAMES');
OK
Time taken: 0.086 seconds
hive>
```

Objective: To make the database as current working database.

Act:

```
USE STUDENTS;
```

Outcome:

```
hive> USE STUDENTS;
OK
Time taken: 0.02 seconds
hive>
```

Objective: To drop database.

Act:

```
DROP DATABASE STUDENTS;
```

Hive provides two kinds of table:

1. Internal or Managed Table
2. External Table

Managed Table

1. Hive stores the Managed tables under the warehouse folder under Hive.
2. The complete life cycle of table and data is managed by Hive.
3. When the internal table is dropped, it drops the data as well as the metadata.

When you create a table in Hive, by default it is internal or managed table. If one needs to create an external table, one will have to use the keyword "EXTERNAL".

Objective: To create managed table named 'STUDENT'.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW
FORMAT DELIMITED FIELDS TERMINATED BY '\t';
```

Objective: To describe the "STUDENT" table.

Act:

```
DESCRIBE STUDENT;
```

Outcome:

```
hive> DESCRIBE STUDENT;
OK
rollno                  int
name                    string
gpa                     float
Time taken: 0.163 seconds, Fetched: 3 row(s)
hive>
```

To check whether an existing table is managed or external, use the below syntax:

DESCRIBE FORMATTED tablename;

It displays complete metadata of a table. You will see one row called table type which will display either MANAGED_TABLE OR EXTERNAL_TABLE.

```
DESCRIBE FORMATTED STUDENT;
```

Objective: To create external table named 'EXT_STUDENT'.

Act:

```
CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t' LOCATION '/STUDENT_INFO;
```

Outcome:

```
hive> CREATE EXTERNAL TABLE IF NOT EXISTS EXT_STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMA
T DELIMITED FIELDS TERMINATED BY '\t'  LOCATION '/STUDENT_INFO';
OK
Time taken: 0.123 seconds
hive>
```

Objective: To load data into the table from file named student.tsv.

Act:

```
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE EXT_STUDENT;
```

Note: Local keyword is used to load the data from the local file system. In this case, the file is copied. To load the data from HDFS, remove local key word from the statement. In this case, the file is moved from the original location.

Outcome:

```
hive> LOAD DATA LOCAL INPATH  '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE EXT_STUDENT;
Loading data to table students.ext_student
Table students.ext_student stats: [numFiles=0, numRows=0, totalSize=0, rawDataSize=0]
OK
Time taken: 5.034 seconds
hive>
```

Objective: To work with collection data types.

Input:

1001,John,Smith:Jones,Mark1!45:Mark2!46:Mark3!43

1002,Jack,Smith:Jones,Mark1!46:Mark2!47:Mark3!42

Act:

```
CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks
MAP<STRING,INT>)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
COLLECTION ITEMS TERMINATED BY ':'
MAP KEYS TERMINATED BY '!';
LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE
STUDENT_INFO;
```

Outcome:

```
hive> CREATE TABLE STUDENT_INFO (rollno INT,name String, sub ARRAY<STRING>,marks MAP<STRING,FLOAT>)
    > ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
    > COLLECTION ITEMS TERMINATED BY ':'
    > MAP KEYS TERMINATED BY '!';
OK
Time taken: 0.112 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/studentinfo.csv' INTO TABLE STUDENT_INFO;
Loading data to table students.student_info
Table students.student_info stats: [numFiles=1, totalSize=109]
OK
Time taken: 0.397 seconds
hive>
```

Objective: To retrieve the student details from "EXT_STUDENT" table.

Act:

```
SELECT * from EXT_STUDENT;
```

Outcome:

```
hive> select * from EXT_STUDENT;
OK
1001    John    3.0
1002    Jack    4.0
1003    Smith   4.5
1004    Scott   4.2
1005    Joshi   3.5
1006    Alex    4.5
1007    David   4.2
1008    James   4.0
1009    John    3.0
1010    Joshi   3.5
Time taken: 0.054 seconds, Fetched: 10 row(s)
hive>
```

Partition is of two types:

1. **STATIC PARTITION:** It is upon the user to mention the partition (the segregation unit) where the data from the file is to be loaded.
2. **DYNAMIC PARTITION:** The user is required to simply state the column, basis which the partitioning will take place. Hive will then create partitions basis the unique values in the column on which partition is to be carried out.

Objective: To create static partition based on "gpa" column.

Act:

```
CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT (rollno INT, name STRING)
PARTITIONED BY (gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED
BY '\t';
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STATIC_PART_STUDENT(rollno INT,name STRING) PARTITIONED BY (gpa FLOAT) RO
W FORMAT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.105 seconds
hive>
```

Objective: To create dynamic partition on column date.

Act:

```
CREATE TABLE IF NOT EXISTS DYNAMIC_PART_STUDENT(rollno INT,name STRING)
PARTITIONED BY (gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED
BY '\t';
```

Bucketing

- Partitioning divides a table into **directories** based on the values of one or more columns

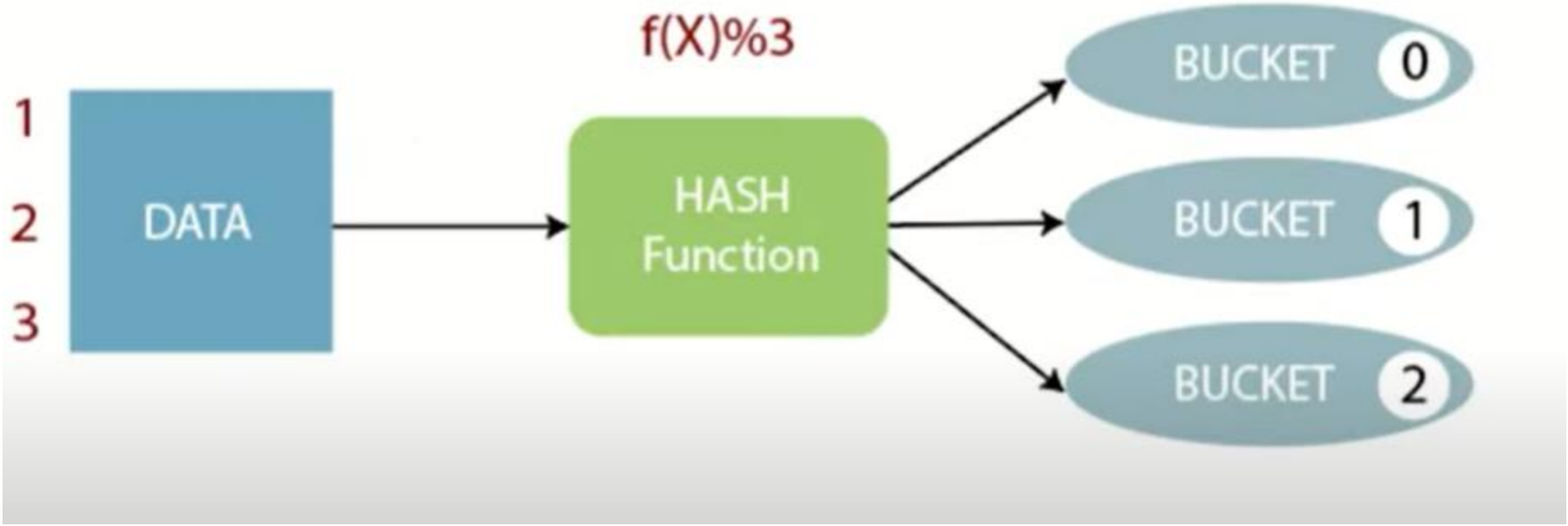
- Each partition is stored as a **separate folder**

/user/hive/warehouse/sales/country=US/
/user/hive/warehouse/sales/country=IN/

- Bucketing splits data within each partition into **fixed number of files** based on the **hash of a column**.

- Buckets are stored as different files inside a directory.

/user/hive/warehouse/students/000000_0
/user/hive/warehouse/students/000001_0
/user/hive/warehouse/students/000002_0
/user/hive/warehouse/students/000003_0


f(X)%3
1
2
3
DATA
HASH
Function
BUCKET 0
BUCKET 1
BUCKET 2

```
hive> create table tab12(id int, name string,sal float)
    > row format delimited
    > fields terminated by ',';
OK
Time taken: 1.723 seconds
hive> load data local inpath '/home/amrit/bucket.txt' into table tab12;
Loading data to table default.tab12
OK
Time taken: 1.644 seconds
hive> select * from tab12;
OK
1       a       222.3
2       b       333.1
3       c       444.1
4       d       555.5
5       e       666.3
6       f       777.2
Time taken: 2.903 seconds, Fetched: 6 row(s)
hive>
```

```
hive> set hive.enforce.bucketing = true;
hive> create table tab12_bucket(id int,name string,sal float)
    > clustered by (id) into 3 buckets
    > row format delimited
    > fields terminated by ',';
OK
Time taken: 0.09 seconds
hive> insert overwrite table tab12_bucket select * from tab12;
```

```
1,a,222.3
2,b,333.1
3,c,444.1
4,d,555.5
5,e,666.3
6,f,777.2
```

```
amrit@amrit-HP-Notebook:~$ hadoop fs -ls /user/hive/warehouse/tab12_bucket
2020-11-25 23:24:57,656 WARN util.NativeCodeLoader: Unable to load native-hadoop
 library for your platform... using builtin-java classes where applicable
Found 3 items
-rw-r--r--   1 amrit supergroup         20 2020-11-25 23:24 /user/hive/warehouse
/tab12_bucket/000000_0
-rw-r--r--   1 amrit supergroup         20 2020-11-25 23:24 /user/hive/warehouse
/tab12_bucket/000001_0
-rw-r--r--   1 amrit supergroup         20 2020-11-25 23:24 /user/hive/warehouse
/tab12_bucket/000002_0
amrit@amrit-HP-Notebook:~$
```

```
amrit@amrit-HP-Notebook:~$ hadoop fs -cat /user/hive/warehouse/tab12_bucket/0000
00_0
2020-11-25 23:25:49,942 WARN util.NativeCodeLoader: Unable to load native-hadoop
 library for your platform... using builtin-java classes where applicable
6,f,777.2
3,c,444.1
amrit@amrit-HP-Notebook:~$ hadoop fs -cat /user/hive/warehouse/tab12_bucket/0000
01_0
2020-11-25 23:26:10,487 WARN util.NativeCodeLoader: Unable to load native-hadoop
 library for your platform... using builtin-java classes where applicable
4,d,555.5
1,a,222.3
amrit@amrit-HP-Notebook:~$ hadoop fs -cat /user/hive/warehouse/tab12_bucket/0000
02_0
2020-11-25 23:26:16,819 WARN util.NativeCodeLoader: Unable to load native-hadoop
 library for your platform... using builtin-java classes where applicable
5,e,666.3
2,b,333.1
amrit@amrit-HP-Notebook:~$
```

Objective: To learn the concept of bucket in hive.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT (rollno INT,name STRING,grade FLOAT)
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' INTO TABLE STUDENT;
```

Set below property to enable bucketing.

```
set hive.enforce.bucketing=true;

// To create a bucketed table having 3 buckets
CREATE TABLE IF NOT EXISTS STUDENT_BUCKET (rollno INT,name STRING,grade
FLOAT)
CLUSTERED BY (grade) into 3 buckets;
// Load data to bucketed table
FROM STUDENT
INSERT OVERWRITE TABLE STUDENT_BUCKET
SELECT rollno,name,grade;
// To display content of first bucket
SELECT DISTINCT GRADE FROM STUDENT_BUCKET
TABLESAMPLE(BUCKET 1 OUT OF 3 ON GRADE);
```

Outcome:

```
hive> CREATE TABLE IF NOT EXISTS STUDENT (rollno INT,name STRING,grade FLOAT)
    > ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.101 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' INTO TABLE STUDENT;
Loading data to table book.student
Table book.student stats: [numFiles=1, totalSize=145]
OK
Time taken: 0.536 seconds
hive>
```

```
hive> set hive.enforce.bucketing=true;
hive>
```

```
hive> CREATE TABLE IF NOT EXISTS STUDENT_BUCKET (rollno INT,name STRING,grade FLOAT)
    > CLUSTERED BY (grade) into 3 buckets;
OK
Time taken: 0.101 seconds
hive>
```

```
hive> FROM STUDENT
    > INSERT OVERWRITE TABLE STUDENT_BUCKET
    > SELECT rollno,name,grade;
```

Views

a **view** is a **virtual table** based on the result of a SELECT query. It does **not store data physically**—it just saves the SQL logic, and every time you query the view, Hive returns the underlying query.

Objective: To create a view table named "STUDENT_VIEW".

Act:

```
CREATE VIEW STUDENT_VIEW AS SELECT rollno, name FROM EXT_STUDENT;
```

Outcome:

```
hive> CREATE VIEW STUDENT_VIEW AS SELECT rollno,name FROM EXT_STUDENT;
OK
Time taken: 0.606 seconds
hive>
```

```
hive> desc sales_tracker;
OK
region                  string
country                 string
item_type               string
sales_channel           string
order_priority          string
order_date              string
order_id                bigint
ship_date               string
units_sold              bigint
unit_price              double
unit_cost               double
total_revenue           double
total_cost              double
total_profit            double
Time taken: 0.253 seconds, Fetched: 14 row(s)
```

Sub-Saharan Africa	Botswana	Clothes	Offline	C	3/8/2015	680517470	3/25/2015	9097	109.28	35.84	994120.16	326036.48	668083.68			
Sub-Saharan Africa	Tanzania	Personal Care	Online	M	6/21/2013	400304734	7/29/2013	7921	81.73	56.67	647383.33	448883.07	198500.26			
Europe	Romania	Office Supplies	Offline	C	1/6/2013	810871112	1/8/2013	3636	651.21	524.96	2367799.56	1908754.56	459045.0			
Sub-Saharan Africa	Mali	Cereal	Online	L	3/17/2012	235702931	4/3/2012	8590	205.7	117.11	1766963.0	1005974.9	760988.1			
Sub-Saharan Africa	Central African Republic	Office Supplies	Offline	C	4/18/2014	668599021	5/12/2014	2163	651.21	524.96	1408567.23	1135488.4	8	273078.75		
Sub-Saharan Africa	Niger	Baby Food	Online	M	1/3/2016	123670709	2/1/2016	5766	255.28	159.42	1471944.48	919215.72	552728.76			
Europe	Austria	Office Supplies	Online	L	5/12/2011	285341823	6/8/2011	7841	651.21	524.96	5106137.61	4116211.36	989926.25			
Asia	India	Fruits	Online	H	7/29/2010	658348691	8/22/2010	8862	9.33	6.92	82682.46	61325.04	21357.42			
Europe	Luxembourg	Baby Food	Offline	L	8/2/2013	817740142	8/19/2013	6335	255.28	159.42	1617198.8	1009925.7	607273.1			
Sub-Saharan Africa	Cape Verde	Beverages	Offline	H	10/23/2013	858877503	11/6/2013	9794	47.45	31.79	464725.3	311351.26	153374.04			
Europe	Sweden	Vegetables	Offline	M	2/5/2017	947434604	2/19/2017	5808	154.06	90.93	894780.48	528121.44	366659.04			
Europe	Iceland	Meat	Offline	H	3/20/2015	869397771	4/17/2015	2975	421.89	364.69	1255122.75	1084952.75	170170.0			
Middle East and North Africa	Qatar	Personal Care	Offline	L	5/6/2012	481065833	5/8/2012	6925	81.73	56.67	565980.25	392439.75	173540.5			
Sub-Saharan Africa	South Sudan	Meat	Online	C	9/30/2013	159050118	10/1/2013	5319	421.89	364.69	2244032.91	1939786.11	304246.8			
Europe	United Kingdom	Office Supplies	Online	M	5/20/2014	350274455	6/14/2014	2850	651.21	524.96	1855948.5	1496136.0	359812.5			
Middle East and North Africa	Tunisia	Cereal	Online	L	4/9/2010	221975171	5/17/2010	6241	205.7	117.11	1283773.7	730883.51	552890.19			
North America	United States of America	Office Supplies	Online	C	6/9/2017	811701095	7/19/2017	9247	651.21	524.96	6021738.87	4854305.12	1	167433.75		
Sub-Saharan Africa	Liberia	Cereal	Online	L	2/8/2015	977313554	3/29/2015	7653	205.7	117.11	1574222.1	896242.83	677979.27			
Sub-Saharan Africa	Eritrea	Snacks	Offline	L	1/25/2010	546986377	2/10/2010	4279	152.58	97.44	652889.82	416945.76	235944.06			
Asia	South Korea	Fruits	Offline	L	3/7/2010	769205892	3/17/2010	3972	9.33	6.92	37058.76	27486.24	9572.52			
Sub-Saharan Africa	Kenya	Clothes	Offline	M	1/3/2013	262770926	2/8/2013	8611	109.28	35.84	941010.08	308618.24	632391.84			
Sub-Saharan Africa	Rwanda	Snacks	Online	M	3/6/2017	866792809	3/18/2017	2109	152.58	97.44	321791.22	205500.96	116290.26			
Central America and the Caribbean	Cuba	Beverages	Offline	C	1/9/2011	890695369	2/23/2011	5408	47.45	31.79	256609.6	171920.32	8	4689.28		
Middle East and North Africa	Libya	Cereal	Offline	M	3/27/2014	964214932	3/31/2014	1480	205.7	117.11	304436.0	173322.8	131113.2			
Europe	Czech Republic	Snacks	Online	C	6/28/2013	887400329	8/17/2013	332	152.58	97.44	50656.56	32350.08	18306.48			
Europe	Montenegro	Beverages	Offline	M	9/4/2011	980612885	9/4/2011	3999	47.45	31.79	189752.55	127128.21	62624.34			
Europe	Montenegro	Clothes	Offline	M	7/14/2016	734526431	8/2/2016	1549	109.28	35.84	169274.72	55516.16	113758.56			
Asia	Philippines	Baby Food	Online	L	2/23/2014	160127294	3/23/2014	4079	255.28	159.42	1041287.12	650274.18	391012.94			
Central America and the Caribbean	El Salvador	Clothes	Offline	L	8/7/2010	238714301	9/13/2010	9721	109.28	35.84	1062310.88	348400.64	7	13910.24		
Australia and Oceania	Tonga	Household	Online	M	1/14/2013	671898782	2/6/2013	8635	668.27	502.54	5770511.45	4339432.9	1431078.55			
Sub-Saharan Africa	Democratic Republic of the Congo	Personal Care	Offline	H	9/30/2010	331604564	11/17/2010	8014	81.73	56.67	654984.22	4	54153.38	200830.84		

```
hive> select region, count(country), avg(total_profit) from sales_tracker group by region;
Query ID = root_20200513001212_cb01727d-7bcf-4370-8bab-2d2115c15843
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1589327438560_0004, Tracking URL = http://quickstart.cloudera:8088/proxy/application_1589327438560_0004/
Kill Command = /usr/lib/hadoop/bin/hadoop job  -kill job_1589327438560_0004
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2020-05-13 00:14:10,220 Stage-1 map = 0%,  reduce = 0%
```

```
OK
Asia    25      349386
Australia and Oceania   12      280747
Central America and the Caribbean       14      371765
Europe  60      465092
Middle East and North Africa    24      422257
North America   8       413719
Sub-Saharan Africa      56      349561
Time taken: 18.285 seconds, Fetched: 7 row(s)
hive>
```

Objective: Querying the view "STUDENT_VIEW".

Act:

```
SELECT * FROM STUDENT_VIEW LIMIT 4;
```

Outcome:

```
hive> SELECT * FROM STUDENT_VIEW LIMIT 4;
OK
1001    John
1002    Jack
1003    Smith
1004    Scott
Time taken: 0.279 seconds, Fetched: 4 row(s)
hive>
```

Objective: To drop the view "STUDENT_VIEW".

Act:

```
DROP VIEW STUDENT_VIEW;
```

Outcome:

```
hive> DROP VIEW STUDENT_VIEW;
OK
Time taken: 0.452 seconds
hive>
```

Sub query

a **subquery** is a query nested inside another query.

Subqueries are used to:
Break down complex logic
Reuse intermediate results
Filter or transform data within the main query

Objective: Write a sub-query to count occurrence of similar words in the file.

Act:

```
CREATE TABLE docs (line STRING);
LOAD DATA LOCAL INPATH '/root/hivedemos/lines.txt' OVERWRITE INTO TABLE docs;
CREATE TABLE word_count AS
SELECT word, count(1) AS count FROM
(SELECT explode (split (line, ' ')) AS word FROM docs) w
GROUP BY word
ORDER BY word;
SELECT * FROM word_count;
```

```
hive> CREATE TABLE docs (line STRING);
OK
Time taken: 0.118 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/lines.txt' OVERWRITE INTO TABLE docs;
Loading data to table students.docs
Table students.docs stats: [numFiles=1, numRows=0, totalSize=91, rawDataSize=0]
OK
Time taken: 2.697 seconds
hive>
```

```
hive> CREATE TABLE word_count AS
    > SELECT word, count(1) AS count FROM
    > (SELECT explode (split (line, ' ')) AS word FROM docs) w
    > GROUP BY word
    > ORDER BY word:

hive> SELECT * FROM word_count;
OK
Hadoop  2
Hive    2
Introducing     1
Introduction    1
Pig     1
Session 3
Welcome 1
to      2
Time taken: 0.062 seconds, Fetched: 8 row(s)
hive>
```

Joins

Objective: To create JOIN between Student and Department tables where we use RollNo from both the tables as the join key.

Act:

```
CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';

LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE STUDENT;

CREATE TABLE IF NOT EXISTS DEPARTMENT(rollno INT,deptno int,name STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\t';

LOAD DATA LOCAL INPATH '/root/hivedemos/department.tsv' OVERWRITE INTO TABLE DEPARTMENT;

SELECT a.rollno, a.name, a.gpa, b.deptno FROM STUDENT a JOIN DEPARTMENT b ON a.rollno = b.rollno;
```

```
hive> CREATE TABLE IF NOT EXISTS STUDENT(rollno INT,name STRING,gpa FLOAT) ROW FORMAT D
ELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.115 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/student.tsv' OVERWRITE INTO TABLE STUDENT
;
Loading data to table students.student
Table students.student stats: [numFiles=1, numRows=0, totalSize=145, rawDataSize=0]
OK
Time taken: 0.723 seconds
hive>
```

```
hive> CREATE TABLE IF NOT EXISTS DEPARTMENT(rollno INT,deptno int,name STRING) ROW FORM
AT DELIMITED FIELDS TERMINATED BY '\t';
OK
Time taken: 0.099 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/department.tsv' OVERWRITE INTO TABLE DEPA
RTMENT;
Loading data to table students.department
Table students.department stats: [numFiles=1, numRows=0, totalSize=120, rawDataSize=0]
OK
Time taken: 0.442 seconds
hive>
```

```
hive> SELECT a.rollno, a.name, a.gpa, b.deptno  FROM STUDENT  a JOIN DEPARTMENT b ON a.
rollno = b.rollno;
OK
1001    John    3.0     101
1002    Jack    4.0     102
1003    Smith   4.5     103
1004    Scott   4.2     104
1005    Joshi   3.5     105
1006    Alex    4.5     101
1007    David   4.2     104
1008    James   4.0     102
Time taken: 115.282 seconds, Fetched: 8 row(s)
hive>
```

```
set hive.auto.convert.join=false;


CREATE TABLE B1(ID INT,NAME STRING,CITY STRING,SAL BIGINT)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

CREATE TABLE B2(ID INT,PID INT,DEPT STRING)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';

LOAD DATA LOCAL INPATH 'C:\Users\apsag\Downloads\data1.txt' INTO TABLE B1;

LOAD DATA LOCAL INPATH 'C:\Users\apsag\Downloads\data2.txt' INTO TABLE B2;

SELECT a.id,a.name,b.pid,b.dept from B1 a join B2 b on(a.id=b.id);
```

Hive supports aggregation functions like avg, count, etc.

Objective: To write the average and count aggregation functions.

Act:

```
SELECT avg(gpa) FROM STUDENT;
SELECT count(*) FROM STUDENT;
```

Outcome:

```
hive> SELECT avg(gpa) FROM STUDENT;

OK
3.839999961853027
Time taken: 28.639 seconds, Fetched: 1 row(s)
hive>

hive> SELECT count(*) FROM STUDENT;

OK
10
Time taken: 26.218 seconds, Fetched: 1 row(s)
hive>
```

Objective: To write group by and having function.

Act:

SELECT rollno, name,gpa FROM STUDENT GROUP BY rollno,name,gpa HAVING gpa > 4.0;

Outcome:

```
1003    Smith   4.5
1004    Scott   4.2
1006    Alex    4.5
1007    David   4.2
Time taken: 78.972 seconds, Fetched: 4 row(s)
hive>
```

RCFile(Record Columnar File)

Act:

```
CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;
INSERT OVERWRITE table STUDENT_RC SELECT * FROM STUDENT;
SELECT SUM(gpa) FROM STUDENT_RC;
```

Outcome:

```
hive> CREATE TABLE STUDENT_RC( rollno int, name string,gpa float ) STORED AS RCFILE;
OK
Time taken: 0.093 seconds
hive>
```

```
hive> INSERT OVERWRITE table STUDENT_RC SELECT * from STUDENT;
```

```
hive> SELECT SUM(gpa) from STUDENT_RC;
```

```
OK
38.39999961853027
Time taken: 25.41 seconds, Fetched: 1 row(s)
hive>
```

SerDe stands for Serializer/Deserializer.

1. Contains the logic to convert unstructured data into records.
2. Implemented using Java.
3. Serializers are used at the time of writing.
4. Deserializers are used at query time (SELECT Statement).

Objective: To manipulate the XML data.

Input:

<employee> <empid>1001</empid> <name>John</name> <designation>Team Lead</designation> </employee>
<employee> <empid>1002</empid> <name>Smith</name> <designation>Analyst</designation> </employee>

Act:

```
CREATE TABLE XMLSAMPLE(xmldata string);
LOAD DATA LOCAL INPATH '/root/hivedemos/input.xml' INTO TABLE XMLSAMPLE;

CREATE TABLE xpath_table AS
SELECT xpath_int(xmldata,'employee/empid'),
xpath_string(xmldata,'employee/name'),
xpath_string(xmldata,'employee/designation')
FROM xmlsample;

SELECT * FROM xpath_table;
```

Outcome:

```
hive> CREATE TABLE XMLSAMPLE(xmldata string);
OK
Time taken: 0.244 seconds
hive>
```

```
hive> LOAD DATA LOCAL INPATH '/root/hivedemos/input.xml' INTO TABLE XMLSAMPLE;
Loading data to table students.xmlsample
Table students.xmlsample stats: [numFiles=1, totalSize=194]
OK
Time taken: 0.889 seconds
hive>
```

```
hive> CREATE TABLE xpath_table AS
    > SELECT xpath_int(xmldata,'employee/empid'),
    > xpath_string(xmldata,'employee/name'),
    > xpath_string(xmldata,'employee/designation')
    > FROM xmlsample;

hive> SELECT * FROM xpath_table;
OK
1001    John    Team Lead
1002    Smith   Analyst
Time taken: 0.064 seconds, Fetched: 2 row(s)
hive>
```

Objective: Write a Hive function to convert the values of a field to uppercase.

Act:

```
package com.example.hive.udf;
import org.apache.hadoop.hive.ql.exec.Description;
import org.apache.hadoop.hive.ql.exec.UDF;
@Description(
 name="SimpleUDFExample")
```

```
 public final class MyLowerCase extends UDF {
  public String evaluate(final String word) {
   return word.toLowerCase();
  }
 }
```

Note: Convert this Java Program into Jar.

```
ADD JAR /root/hivedemos/UpperCase.jar;
CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
SELECT TOUPPERCASE(name) FROM STUDENT;
```

Outcome:

```
hive> ADD JAR /root/hivedemos/UpperCase.jar;
Added [/root/hivedemos/UpperCase.jar] to class path
Added resources: [/root/hivedemos/UpperCase.jar]
hive> CREATE TEMPORARY FUNCTION touppercase AS 'com.example.hive.udf.MyUpperCase';
OK
Time taken: 0.014 seconds
hive>
```

```
hive> Select touppercase (name) from STUDENT;
OK
JOHN
JACK
SMITH
SCOTT
JOSHI
ALEX
DAVID
JAMES
JOHN
JOSHI
Time taken: 0.061 seconds, Fetched: 10 row(s)
hive>
```

Introduction to Pig

What is Pig

Apache Pig is a platform for data analysis. It is an alternative to MapReduce Programming. Pig was developed as a research project at Yahoo.

Key Features of Pig

1. It provides an **engine** for executing **data flows** (how your data should flow). Pig processes data in parallel on the Hadoop cluster.
2. It provides a language called "**Pig Latin**" to express data flows.
3. Pig Latin contains operators for many of the traditional data operations such as join, filter, sort, etc.
4. It allows users to develop their own functions (User Defined Functions) for reading, processing, and writing data.

THE ANATOMY OF PIG

The main components of Pig are as follows:

1. Data flow language (**Pig Latin**).
2. Interactive shell where you can type Pig Latin statements (**Grunt**).
3. Pig interpreter and execution engine.

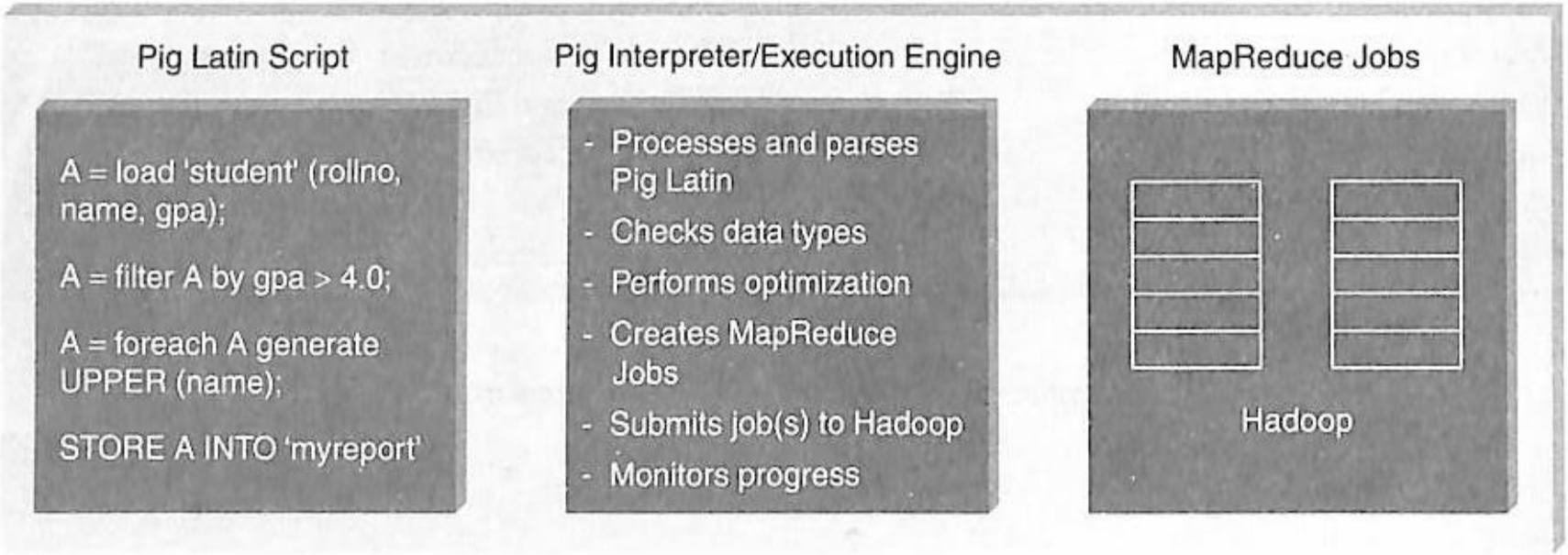


Figure 10.1 The anatomy of Pig.

PIG ON HADOOP

Pig runs on Hadoop. Pig uses both Hadoop Distributed File System and MapReduce Programming. By default, Pig reads input files from HDFS. Pig stores the intermediate data (data produced by MapReduce jobs) and the output in HDFS. However, Pig can also read input from and place output to other sources.

Pig supports the following:

1. HDFS commands.
2. UNIX shell commands.
3. Relational operators.
4. Positional parameters.
5. Common mathematical functions.
6. Custom functions.
7. Complex data structures.

1. **Pigs Eat Anything:** Pig can process different kinds of data such as structured and unstructured data.
2. **Pigs Live Anywhere:** Pig not only processes files in HDFS, it also processes files in other sources such as files in the local file system.
3. **Pigs are Domestic Animals:** Pig allows you to develop user-defined functions and the same can be included in the script for complex operations.
4. **Pigs Fly:** Pig processes data quickly.

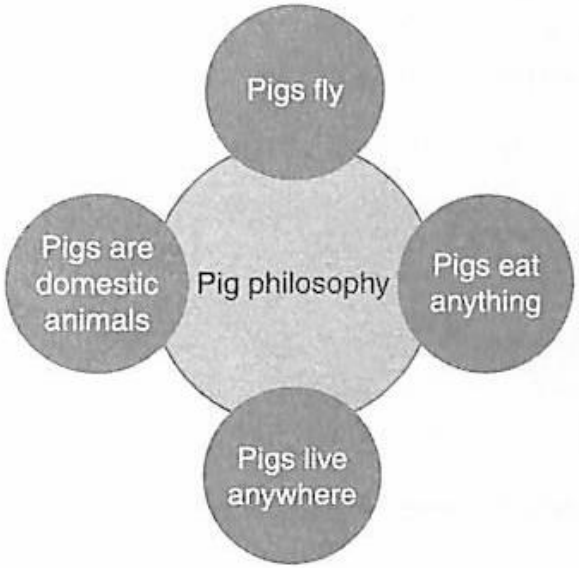


Figure 10.2 Pig philosophy.

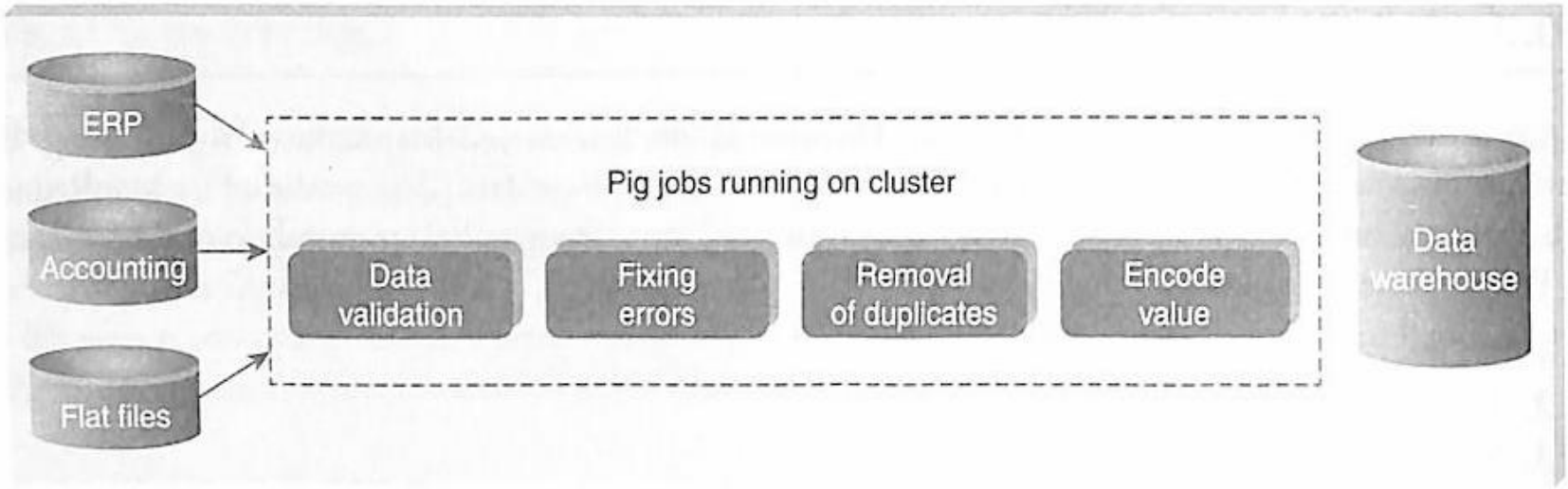


Figure 10.3 Pig: ETL Processing.

Pig is widely used for "ETL" (Extract, Transform, and Load).

Pig Latin Statements

1. Pig Latin statements are basic constructs to process data using Pig.
2. Pig Latin statement is an operator.
3. An operator in Pig Latin takes a relation as input and yields another relation as output.
4. Pig Latin statements include schemas and expressions to process data.
5. Pig Latin statements should end with a semi-colon.

Pig Latin Statements are generally ordered as follows:

1. **LOAD** statement that reads data from the file system.
2. Series of statements to perform transformations.
3. **DUMP** or **STORE** to display/store result.

The following is a simple Pig Latin script to load, filter, and store "student" data.

```
A = load 'student' (rollno, name, gpa);
A = filter A by gpa > 4.0;
A = foreach A generate UPPER (name);
STORE A INTO 'myreport'
```

Note: In the above example **A** is a relation and NOT a variable.

Table 10.1 Valid and invalid identifiers

Valid Identifier	**Y**	**A1**	**A1_2014**	**Sample**
Invalid Identifier	**5**	**Sales$**	**Sales%**	**_Sales**

Pig Latin: Comments

In Pig Latin two types of comments are supported:

1. Single line comments that begin with "--".
2. Multiline comments that begin with "/* and end with */".

Pig Latin: Case Sensitivity

1. Keywords are *not* case sensitive such as LOAD, STORE, GROUP, FOREACH, DUMP, etc.
2. Relations and paths are case-sensitive.
3. Function names are case sensitive such as PigStorage, COUNT.

Table 10.2 Operators in Pig Latin

Arithmetic	Comparison	Null	Boolean
+	==	IS NULL	AND
–	!=	IS NOT NULL	OR
*	<		NOT
/	>		
%	<=		
	>=		

Table 10.3 Simple data types supported in Pig

Name	Description
Int	Whole numbers
Long	Large whole numbers
Float	Decimals
Double	Very precise decimals
Chararray	Text strings
Bytearray	Raw bytes
Datetime	Datetime
Boolean	true or false

Table 10.4 Complex data types in Pig

Name	Description
Tuple	An ordered set of fields. **Example: (2,3)**
Bag	A collection of tuples. **Example: {(2,3),(7,5)}**
map	key, value pair **(open # Apache)**

RUNNING PIG

run Pig in two ways:

1. Interactive Mode.
2. Batch Mode.

Interactive Mode

You can run Pig in interactive mode by invoking **grunt shell**. Type pig to get grunt shell as shown below.

```
root@volgalnx010:~
[root@volgalnx010 ~]# pig
```

```
root@volgalnx010:~
[root@volgalnx010 ~]# pig
2015-02-23 21:07:38,916 [main] INFO  org.apache.pig.Main - Apache Pig version 0.12.0-cdh5.
1.3 (rexported) compiled Sep 16 2014, 20:39:43
2015-02-23 21:07:38,917 [main] INFO  org.apache.pig.Main - Logging error messages to: /roo
t/pig_1424705858915.log
2015-02-23 21:07:38,934 [main] INFO  org.apache.pig.impl.util.Utils - Default bootup file
/root/.pigbootup not found
2015-02-23 21:07:39,313 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - ma
pred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2015-02-23 21:07:39,313 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs
.default.name is deprecated. Instead, use fs.defaultFS
2015-02-23 21:07:39,313 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecut
ionEngine - Connecting to hadoop file system at: hdfs://volgalnx010.ad.infosys.com:9000
2015-02-23 21:07:39,800 [main] WARN  org.apache.hadoop.util.NativeCodeLoader - Unable to l
oad native-hadoop library for your platform... using builtin-java classes where applicable
2015-02-23 21:07:40,234 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs
.default.name is deprecated. Instead, use fs.defaultFS
grunt>
```

Once you get the grunt prompt, you can type the Pig Latin statement as shown below.

```
grunt> A = load '/pigdemo/student.tsv' as (rollno, name, gpa);
grunt> DUMP A;
```

Here, the path refers to HDFS path and DUMP displays the result on the console as shown below.

```
(1001,John,3.0)
(1002,Jack,4.0)
(1003,Smith,4.5)
(1004,Scott,4.2)
(1005,Joshi,3.5)
grunt>
```

Batch Mode

You need to create **"Pig Script"** to run pig in batch mode. Write Pig Latin statements in a file and save it with **.pig** extension.

EXECUTION MODES OF PIG

You can execute pig in two modes:

1. Local Mode.
2. MapReduce Mode.

Local Mode

To run pig in local mode, you need to have your files in the local file system.

Syntax:

```
pig –x local filename
```

MapReduce Mode

To run pig in MapReduce mode, you need to have access to a Hadoop Cluster to read /write file. This is the default mode of Pig.

Syntax:

```
pig filename
```

all HDFS commands in Grunt shell. For example, you can create a directory as shown below.

```
grunt> fs -mkdir /piglatindemos;
grunt>
```

1

FILTER operator is used to select tuples from a relation based on specified conditions.

Objective: Find the tuples of those student where the GPA is greater than 4.0.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = filter A by gpa > 4.0;
DUMP B;
```

Output:

```
(1003,Smith,4.5)
(1004,Scott,4.2)
[root@volgalnx010 pigdemos]#
```

2

Use ***FOREACH*** when you want to do data transformation based on columns of data.

Objective: Display the name of all students in uppercase.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = foreach A generate UPPER (name);
DUMP B;
```

Output:

```
(JOHN)
(JACK)
(SMITH)
(SCOTT)
(JOSHI)
[root@volgalnx010 pigdemos]#
```

3

GROUP operator is used to group data.

Objective: Group tuples of students based on their GPA.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = GROUP A BY gpa;
DUMP B;
```

Output:

```
(3.0,{(1001,John,3.0),(1001,John,3.0)})
(3.5,{(1005,Joshi,3.5),(1005,Joshi,3.5)})
(4.0,{(1008,James,4.0),(1002,Jack,4.0)})
(4.2,{(1007,David,4.2),(1004,Scott,4.2)})
(4.5,{(1006,Alex,4.5),(1003,Smith,4.5)})
[root@volgalnx010 pigdemos]#
```

DISTINCT operator is used to remove duplicate tuples. In Pig, DISTINCT operator works on the entire tuple and NOT on individual fields.

Objective: To remove duplicate tuples of students.

Input:

Student (rollno:int,name:chararray,gpa:float)

Input:

1001	John	3.0
1002	Jack	4.0
1003	Smith	4.5
1004	Scott	4.2
1005	Joshi	3.5
1006	Alex	4.5
1007	David	4.2
1008	James	4.0
1001	John	3.0
1005	Joshi	3.5

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = DISTINCT A;
DUMP B;
```

Output:

```
(1001,John,3.0)
(1002,Jack,4.0)
(1003,Smith,4.5)
(1004,Scott,4.2)
(1005,Joshi,3.5)
(1006,Alex,4.5)
(1007,David,4.2)
(1008,James,4.0)
[root@volgalnx010 pigdemos]#
```

LIMIT operator is used to limit the number of output tuples.

Objective: Display the first 3 tuples from the "student" relation.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = LIMIT A 3;
DUMP B;
```

Output:

```
(1001,John,3.0)
(1002,Jack,4.0)
(1003,Smith,4.5)
[root@volgalnx010 pigdemos]#
```

ORDER BY is used to sort a relation based on specific value.

Objective: Display the names of the students in Ascending Order.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = ORDER A BY name;
DUMP B;
```

Output:

```
(1006,Alex,4.5)
(1007,David,4.2)
(1002,Jack,4.0)
(1008,James,4.0)
(1001,John,3.0)
(1001,John,3.0)
(1005,Joshi,3.5)
(1005,Joshi,3.5)
(1004,Scott,4.2)
(1003,Smith,4.5)
[root@volgalnx010 pigdemos]#
```

Objective: To join two relations namely, "student" and "department" based on the values contained in the "rollno" column.

Input:

Student (rollno:int,name:chararray,gpa:float)

Department(rollno:int,deptno:int,deptname:chararray)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = load '/pigdemo/department.tsv' as (rollno:int, deptno:int,deptname:chararray);
C = JOIN A BY rollno, B BY rollno;
DUMP C;
DUMP B;
```

Output:

```
(1001,John,3.0,1001,101,B.E.)
(1001,John,3.0,1001,101,B.E.)
(1002,Jack,4.0,1002,102,B.Tech)
(1003,Smith,4.5,1003,103,M.Tech)
(1004,Scott,4.2,1004,104,MCA)
(1005,Joshi,3.5,1005,105,MBA)
(1005,Joshi,3.5,1005,105,MBA)
(1006,Alex,4.5,1006,101,B.E)
(1007,David,4.2,1007,104,MCA)
(1008,James,4.0,1008,102,B.Tech)
[root@volgalnx010 pigdemos]#
```

Objective: To merge the contents of two relations "student" and "department".

Input:

Student (rollno:int,name:chararray,gpa:float)

Department(rollno:int,deptno:int,deptname:chararray)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno, name, gp);
B = load '/pigdemo/department.tsv' as (rollno, deptno,deptname);
C = UNION A,B;
STORE C INTO '/pigdemo/uniondemo';
DUMP B;
```

Output:

"Store" is used to save the output to a specified path. The output is stored in two files: part-m-00000 contains "student" content and part-m-00001 contains "department" content.

Name	Type	Size	Replication	Block Size	Modification Time	Permission	Owner	Group
_SUCCESS	file	0 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup
part-m-00000	file	146 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup
part-m-00001	file	114 B	3	128 MB	2015-02-24 17:23	rw-r--r--	root	supergroup

Objective: To partition a relation based on the GPAs acquired by the students.

- GPA = 4.0, place it into relation X.
- GPA is < 4.0, place it into relation Y.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
SPLIT A INTO X IF gpa==4.0, Y IF gpa<=4.0;
DUMP X;
```

Output: Relation X

```
(1002,Jack,4.0)
(1008,James,4.0)
[root@volgalnx010 pigdemos]#
```

Output: Relation Y

```
(1001,John,3.0)
(1002,Jack,4.0)
(1005,Joshi,3.5)
(1008,James,4.0)
(1001,John,3.0)
(1005,Joshi,3.5)
[root@volgalnx010 pigdemos]#
```

Objective: To depict the use of ***SAMPLE.***

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = SAMPLE A 0.01;
DUMP B;
```

Objective: To calculate the average marks for each student.

Input:

Student (studname:chararray,marks:int)

Act:

```
A = load '/pigdemo/student.csv' USING PigStorage (',') as (studname:chararray,marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE A.studname, AVG(A.marks);
DUMP C;
```

Output:

```
({(Jack),(Jack),(Jack),(Jack)},39.75)
({(John),(John),(John),(John)},39.0)
[root@volgalnx010 pigdemos]#
```

Note: You need to use PigStorage function if you wish to manipulate files other than .tsv.

MAX is used to compute the maximum of numeric values in a single column bag.

Objective: To calculate the maximum marks for each student.
Input:
Student (studname:chararray,marks:int)
Act:

```
A = load '/pigdemo/student.csv' USING PigStorage (',') as (studname:chararray, marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE A.studname, MAX(A.marks);
DUMP C;
```

Output:

```
({(Jack),(Jack),(Jack),(Jack)},46)
({(John),(John),(John),(John)},45)
[root@volgalnx010 pigdemos]#
```

Note: Similarly, you can try the MIN and the SUM functions as well.

Objective: To count the number of tuples in a bag.

Input:

Student (studname:chararray,marks:int)

Act:

```
A = load '/pigdemo/student.csv' USING PigStorage (',') as (studname:chararray, marks:int);
B = GROUP A BY studname;
C = FOREACH B GENERATE  A. studname,COUNT(A);
DUMP C;
```

Output:

```
({(Jack),(Jack),(Jack),(Jack)},4)
({(John),(John),(John),(John)},4)
[root@volgalnx010 pigdemos]#
```

Note: The default file format of Pig is .tsv file. Use PigStorage() to manipulate files other than .tsv file.

Objective: To use the complex data type "Tuple" to load data.

Input:

(John,12)	**(Jack,13)**
(James,7)	**(Joseph,5)**
(Smith,8)	**(Scott,12)**

Act:

```
A = LOAD '/root/pigdemos/studentdata.tsv' AS (t1:tuple(t1a:chararray,
t1b:int),t2:tuple(t2a:chararray,t2b:int));
B = FOREACH A GENERATE t1.t1a, t1.t1b,t2.$0,t2.$1;
DUMP B;
```

Output:

```
(John,12,Jack,13)
(James,7,Joseph,5)
(Smith,8,Scott,12)
[root@volgalnx010 pigdemos]#
```

Note: You can refer to the field using Positional Notation as shown above. The Positional Notation is denoted by $ sign and the position starts with 0 (e.g., $0).

MAP represents a key/value pair.

Objective: To depict the complex data type "map".

Input:

John	**[city#Bangalore]**
Jack	**[city#Pune]**
James	**[city#Chennai]**

Act:

```
A = load '/root/pigdemos/studentcity.tsv' Using PigStorage as
(studname:chararray,m:map[chararray]);

B = foreach A generate m#'city' as CityName:chararray;

DUMP B
```

Output:

```
(Bangalore)
(Pune)
(Chennai)
[root@volgalnx010 pigdemos]#
```

Pig user can use Piggy Bank functions in Pig Latin script and they can also share their functions in Piggy Bank.

Objective: To use Piggy Bank string UPPER function.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
register '/root/pigdemos/piggybank-0.12.0.jar';
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
upper = foreach A generate
org.apache.pig.piggybank.evaluation.string.UPPER(name);
DUMP upper;
```

Output:

Note: You need to use the "**register**" keyword to use Piggy Bank jar function in your pig script.

Pig allows you to create your own function for complex analysis.

Objective: To depict user-defined function.

Java Code to convert name into uppercase:

```
package myudfs;
import java.io.IOException;
import org.apache.pig.EvalFunc;
import org.apache.pig.data.Tuple;
import org.apache.pig.impl.util.WrappedIOException;
public class UPPER extends EvalFunc<String>
  {
    public String exec(Tuple input) throws IOException {
      if (input == null || input.size() == 0)
        return null;
        try{
          String str = (String)input.get(0);
          return str.toUpperCase();
        }catch(Exception e){
          throw WrappedIOException.wrap ("Caught exception processing input row ", e);
      }
    }
  }
```

Note: Convert above java class into jar to include this function into your code.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
register /root/pigdemos/myudfs.jar;
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
B = FOREACH A GENERATE myudfs.UPPER(name);
DUMP B;
```

Output:

```
(JOHN)
(JACK)
(SMITH)
(SCOTT)
(JOSHI)
(ALEX)
(DAVID)
(JAMES)
(JOHN)
(JOSHI)
[root@volgalnx010 pigdemos]#
```

Pig allows you to pass parameters at runtime.

Objective: To depict parameter substitution.

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '$student' as (rollno:int, name:chararray, gpa:float);
DUMP A;
```

Execute:

```
pig –param student=/pigdemo/student.tsv parameterdemo.pig
```

Output:

```
(1001,John,3.0)
(1002,Jack,4.0)
(1003,Smith,4.5)
(1004,Scott,4.2)
(1005,Joshi,3.5)
(1006,Alex,4.5)
(1007,David,4.2)
(1008,James,4.0)
(1001,John,3.0)
(1005,Joshi,3.5)
[root@volgalnx010 pigdemos]#
```

Objective: To depict the use of ***DESCRIBE.***

Input:

Student (rollno:int,name:chararray,gpa:float)

Act:

```
A = load '/pigdemo/student.tsv' as (rollno:int, name:chararray, gpa:float);
DESCRIBE A;
```

Output:

```
A: {rollno: int,name: chararray,gpa: float}
[root@volgalnx010 pigdemos]#
```

Objective: To count the occurrence of similar words in a file.

Input:

Welcome to Hadoop Session

Introduction to Hadoop

Introducing Hive

Hive Session

Pig Session

Act:

```
lines = LOAD '/root/pigdemos/lines.txt' AS (line:chararray);
words = FOREACH lines GENERATE FLATTEN(TOKENIZE(line)) as word;
grouped = GROUP words BY word;
wordcount = FOREACH grouped GENERATE group, COUNT(words);
DUMP wordcount;
```

Output:

```
(to,2)
(Pig,1)
(Hive,2)
(Hadoop,2)
(Session,3)
(Welcome,1)
(Introducing,1)
(Introduction,1)
[root@volgalnx010 pigdemos]#
```